

Session 7 · Deck 7.B2

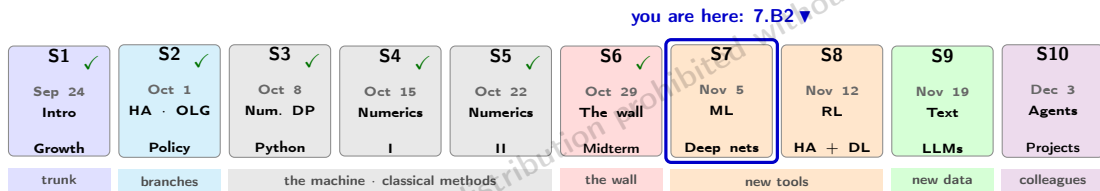
Deep VFI and Actor–Critic

Zhigang Feng

Thursday, November 5, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are



Build it on paper → solve it on the machine → push it to the frontier with AI

Block B: 7.B1 the model as a loss function · **7.B2** deep VFI and actor-critic · 7.B3 structure and validation.

The Other Road

7.B1 required first-order conditions — a real restriction, and it bites exactly where macroeconomics gets interesting:

- **occasionally binding constraints** — the Euler equation holds with an inequality;
- **discrete choice** — default, retirement, entry: no derivative to set to zero;
- **kinks** — a borrowing limit, an irreversibility.

So go back one step, to the object the FOCs were derived from:

$$V(k) = \max_{k'} \{u(c) + \beta V(k')\},$$

also a functional equation with a residual. **Make the residual small and you have solved it** — and **no derivatives of the model are needed**, only derivatives of the *network*.

The price is the max, which now sits *inside* the loss.

Feng & Santos, "Markov Equilibria with Quasi-Hyperbolic Discounting", working paper.

ECON 282E · Session 7 · Deck 7.B2

Research in the wild

The route with theory attached. Feng & Santos construct Markov equilibrium for a time-inconsistent economy from **value-function operators**, and obtain existence *constructively* — as the limit of a sequence of ε -equilibria. Where an economy is hard enough that **existence is itself in question**, the value function is where the argument gets made. That is a reason to prefer this road beyond convenience.

Replace the Grid with a Network

Instead of storing $V(k)$ at grid points, approximate both unknowns:

$$V(k) \approx \mathcal{V}(k; \theta_v), \quad g(k) \approx \mathcal{G}(k; \theta_g).$$

The loop keeps classical VFI's rhythm exactly — *evaluate the current policy, then improve it* — with two substitutions:

1. **sample** states $k \sim D(k)$ instead of laying down a grid;
2. **fit** the networks by gradient descent instead of writing into a table.

Why it scales

A network handles high-dimensional k without a mesh: states are sampled, the network **generalises** between them, and the parameter count grows polynomially in d rather than as N^d .

“Generalises between them” is doing the work, and it is also the risk. A grid makes no claim about the points it does not contain. **A network always answers**, everywhere, whether or not it should.

Deep VFI, the Whole Loop in One Place

Deep value function iteration

Represent $V \approx \mathcal{V}(\cdot; \theta_v)$ and $g \approx \mathcal{G}(\cdot; \theta_g)$, and iterate:

1. **Sample** a batch of states $\{k_i\} \sim D(k)$ — no grid.
2. **Value step (the critic)**. Fit $\mathcal{V}$ by least squares to the Bellman target $\hat{V}(k) = u(k, g(k)) + \beta \mathcal{V}(f(k, g(k)))$.
3. **Policy step (the actor)**. Improve $\mathcal{G}$ by gradient ascent on $u(k, g(k)) + \beta \mathcal{V}(f(k, g(k)))$.
4. **Repeat** until both networks stop moving.

Steps 2 and 3 are **policy evaluation** and **policy improvement** — 1.B2's two halves of value-function iteration, unchanged in logic and wholly changed in machinery.

Notice what is *not* in step 3: an $\arg \max$. The policy network is nudged in the direction that raises the objective, rather than being resolved at each state.

Deep VFI Against Classical VFI

	Classical VFI (3.A2)	Deep VFI
state space	a fixed grid $\{k_1, \dots, k_N\}$	a sampled batch $k_i \sim D(k)$
representation	table, plus interpolation	a network $\mathcal{V}(k; \theta_v)$
value update	exact $\max_{k'}$ at each point	least-squares fit to $\hat{V}$
policy	$\arg \max$ by grid search	a learned network, by ascent
expectation	quadrature on a grid	exact sum, or Monte Carlo
cost in d	N^d	polynomial
convergence	a contraction (Banach)	no general guarantee

The trade, in one line. Give up the exact maximisation and the contraction guarantee; gain dimension, and the ability to fit scattered, simulated data.

Read the last row slowly. 3.A2's error bound $\|V_n - V^*\| \leq \beta^n \|V_0 - V^*\| / (1 - \beta)$ told you *how far you were* without knowing the answer. **Nothing here replaces it.**

Losing the bound is why 7.B3 exists.

The Value Step, and the Detach That Makes It Work

Hold the policy $g^{(n-1)}$ fixed. The one-step Bellman target is

$$\hat{V}(k) = u(k, g^{(n-1)}(k)) + \beta \mathcal{V}^{(n-1)}(f(k, g^{(n-1)}(k))),$$

and the value network is fitted to it by least squares:

$$\min_{\theta_v} \mathbb{E}_{k \sim D} \left[(\mathcal{V}(k; \theta_v) - \hat{V}(k))^2 \right].$$

This is supervised learning again, with the “label” at each sampled state being the Bellman target.

`detach()` the target

$\hat{V}$ contains $\mathcal{V}^{(n-1)}$, which is the *same network*. If the gradient is allowed to flow into the target as well as the prediction, the optimizer can lower the loss by **moving the target** rather than fitting it. Detaching freezes the target for the step. It is one line of code and the difference between converging and not.

The idea has a name in deep RL — the **target network** — and it returns in S8.

The Policy Step, and Why Two Networks

Where is the maximisation actually solved?

grid VFI (3.A2)	at every grid point
projection (5.B6)	at the collocation nodes only
deep VFI	at every sampled state, every batch

What makes it hard is not that a method is classical, but whether the approximant **preserves concavity**. A concave interpolant leaves a concave program with one maximiser, which 4.A2's golden section finds every time; a rippling polynomial — or a generic network — does not.

So let a second network **output the action directly**, $k' = \mathcal{G}(k; \theta_g)$, improved by gradient *ascent* on $u(k, \mathcal{G}(k)) + \beta \mathcal{V}(f(k, \mathcal{G}(k)))$.

The two roles

The actor $\mathcal{G}$ — the doer. State in, action out; it never maximises anything, it is *improved*. **The**

critic $\mathcal{V}$ — the judge. State in, value out; it scores what the actor did. The actor proposes, the critic evaluates, both improve together.

Actor-critic does not avoid the maximisation — it *amortises* it. The actor learns the *arg max* as a *function of the state*: one gradient step over parameters instead of one exact maximisation everywhere. Which is why 7.B3 matters: **restore concavity and the inner problem is well behaved again.**

Two Moving Targets

The awkwardness of the scheme is that **neither network is chasing a fixed object**.

- The critic is fitted to a target built from the *current* policy. Improve the policy and the target moves.
- The actor ascends an objective built from the *current* critic. Refit the critic and the landscape moves.

Classical VFI has this too — but there the value step is a **contraction**, which is precisely what makes the alternation converge. Replace the exact update with a least-squares fit on sampled states and the contraction is gone.

What is done in practice

Freeze and sync. Hold a frozen copy of the target network and refresh it every n iterations, so each side chases something stationary for a while. **Unequal step sizes.** Let the critic learn faster than the actor, so the judge is roughly right before the doer trusts it.

Both devices reappear in S8's frontier solver, where the frozen copy is what imposes *perceived* = *actual* in equilibrium.

One Step or Many: a Bias–Variance Choice

One step. $\hat{V}(k_i) = u(k_i, g(k_i)) + \beta \mathcal{V}(k_{i,1})$.

Low variance — one transition. *High bias* — it leans almost entirely on the current $\mathcal{V}$, which is wrong early on.

T steps.

$$\hat{V}(k_i) = \sum_{t=0}^{T-1} \beta^t u(k_{i,t}, g(k_{i,t})) + \beta^T \mathcal{V}(k_{i,T}).$$

Lower bias — T periods of real simulated reward, the network used only for the tail. *Higher variance* — paths differ.

The dial

T trades the network's error against sampling noise. Typical values are 30–50: long enough for β^T to make the tail nearly irrelevant, short enough that the paths have not dispersed.

At $\beta = 0.99$ quarterly, $\beta^{50} \approx 0.61$ and $\beta^{200} \approx 0.13$: **the discount factor sets how much horizon must be simulated** before the critic's error stops mattering. S8 names the family — $T = 1$ is temporal difference, $T = \infty$ is Monte Carlo, in between is $\text{TD}(\lambda)$.

Where to Sample, and Why Not Chebyshev

Chebyshev and finite elements must approximate over the **entire** domain — that is what a basis expansion is. As d grows, covering the whole domain is exactly what becomes impossible.

Sampling from the model's own **stationary distribution** does something different in kind: it spends the approximation budget on the region the economy actually visits, and ignores the rest.

That is the synergy. **Monte Carlo generates data where it matters; the network fits scattered, mesh-free data.** Neither half works without the other.

Three concerns, and their remedies

Rare but important states. A crisis is off the ergodic path by construction — mix in *targeted* or importance sampling. **Multiple steady states.** One stationary distribution can miss an entire basin. **Out-of-distribution behaviour.** The network answers confidently outside the region it saw. **Stress-test separately.**

7.B1 measured the first of these: two decades of accuracy lost at the edge of the sampled band.

Equilibrium Conditions Can Be Built In

A network's output can be made to satisfy economic restrictions rather than merely approach them. Four devices, in increasing order of strength:

1. **Output transformation** — a sigmoid bounds the action into the feasible set. 7.B1 used it.
2. **Penalty** — add $\lambda \sum (\text{violation})^2$ to the loss.
3. **Lagrangian** — carry dual variables and update them during training.
4. **Architecture** — structure the network so an identity holds automatically.

Device (4) is 7.B3.

Which to reach for

An identity — market clearing, a budget constraint — should be imposed by (1) or (4). A quantity that must equal another should never be *learned*. **An inequality** that binds sometimes needs (2) or (3), or a Fischer–Burmeister reformulation.

A penalty coefficient is a real hyperparameter with real consequences: too small and the constraint is violated, too large and it dominates the residual and nothing else is learned.

Macroeconomics in Reinforcement-Learning Vocabulary

RL	Economics	Note
agent	household or firm	the optimiser
environment	prices and constraints	the rules of the game
state S	(k, z, Γ)	wealth, shocks, the distribution
action A	c, i, l	the choice
transition P	$k' = (1 - \delta)k + i$ $\Gamma' = H(\Gamma)$	micro: known exactly macro: often unknown
reward R	$u(c)$	period utility
discount γ	β	time preference

The two transition rows are the whole story of S8.

We know the household's law of motion exactly, which is why nothing in this deck needed exploration.

We do *not* know H . In Krusell–Smith it is guessed and regressed; the frontier replaces the guess with a network that takes Γ itself as an input.

2.B1 promised this frame.

Two Practical Problems, and Their Standard Answers

The horizon is infinite; a simulation is not.

Truncate at T and close with the critic:

$$\sum_{t=0}^{T-1} \beta^t u_t + \beta^T \mathcal{V}(k_T).$$

The tail is not dropped — it is *estimated*, and β^T bounds the damage. The critic exists for exactly this.

The expectation is unknown in closed form.

Simulate. Draw shock paths, average the realised returns, and let the noise be absorbed by the same stochastic descent that already tolerates mini-batch noise.

Where the shock is a finite Markov chain — as in 7.B1 — **take the exact sum instead**. It costs one matrix product and removes a whole source of error.

A rule worth carrying: never sample what you can sum. The inner expectation is contamination of the residual itself, and unlike outer sampling noise it buys you nothing.

Euler or Bellman? A Decision, Not a Doctrine

	Euler residual (7.B1)	Deep VFI / actor-critic (7.B2)
networks	one	two
requires	first-order conditions	only the Bellman equation
flavour	supervised	reinforcement
gives you	the policy	the policy and the value
handles kinks	badly	naturally
discrete choice	no	yes
accuracy when FOCs exist	higher	lower

Use the Euler road when the FOCs are clean: one network, a sharper residual, and a quantity you can audit directly against a classical solve.

Use the Bellman road when they are not — and accept that you have given up the cleanest validation instrument along with them. 7.B3 supplies a replacement.

The value function is worth having in its own right — welfare comparisons need it, and Euler methods do not produce one.

Two Questions This Deck Cannot Answer

Why sample along the state space at all?

Because accuracy is only worth buying where the economy actually goes. But the ergodic distribution **depends on the policy being learned** — so the sampler and the solution are a *joint fixed point*, and each is chasing the other.

Train only where the current policy goes and it can settle in a corner, satisfy the Bellman equation beautifully there, and be wrong everywhere else — with a small loss reporting success. **That is on-policy against off-policy.**

Both have the same shape: **the object being estimated depends on the policy doing the estimating.** Nothing here resolves that; the methods above work anyway because the growth model is forgiving. **Session 8 is where it stops being forgiving.**

What goes wrong in a T -step rollout?

Uncertainty unfolds. Every simulated step multiplies the branches the path could have taken, so the variance of the return estimate **compounds with T** — while the bias from the critic falls.

Longer horizons buy less bias and pay in variance.

Controlling that exchange — baselines, temporal-difference targets, variance reduction — is most of what reinforcement learning is.

Takeaways, and What 7.B3 Has to Fix

- The Bellman equation is a functional equation with a residual, so it is a loss — and it needs **no first-order conditions**.
- The inner **max** is the obstacle, and **a second network removes it**: the actor acts, the critic judges.
- The value target must be **detached**, or the optimizer lowers the loss by moving the target.
- Sampling from the ergodic region is what makes this scale — and is exactly why the answer **outside** that region is unconstrained.
- **The contraction is gone**. No error bound survives into this deck.

Bridge to 7.B3

Two things are now owed. **First**, the networks obey no economics: nothing makes a value function concave or a policy monotone, though theory says both. **ICNN** and monotone networks impose it by construction. **Second**, with no contraction bound left, **how do you know any of this worked?**